

# User-Verifiable Receipt Interfaces for Anonymous-DHT Claims

## A Minimal Discoverability and Validation Contract (Series Synthesis)

Unified archive note v0.08 (2026-02-28; archive v129)

### Abstract

Most mechanism papers in this archive culminate in a published receipt payload: an ABOM/OINL envelope plus a small set of line items, anchored in an append-only transparency log.[2, 3, 5] A natural question is what an *end user* can verify while interacting with a live system. This note does not attempt to solve “trustless anonymity”; it standardizes a small *user-verifiability interface* that is sufficient to (i) discover the relevant receipt, and (ii) validate that the receipt is anchored in an append-only log with inclusion/consistency proofs. The goal is to make the gap between provider-side accountability and user-side assurance explicit and machine-addressable, without bloating mechanism papers.

**Scope.** This is a short interface note. It specifies the minimal discoverability and validation hooks a deployment can expose so that users can *locate* and *validate* published claims. It does not cover external probing of a dishonest provider, censorship-resistance of publication, or client-side traffic analysis; those are separate threat models.

If the deployment uses OHTTP-style gateways and wants standardized *transport* discovery, RFC 9540 (SVCB/HTTPS parameters for oblivious services) can be used; the UVI record remains a higher-layer pointer to the published receipt.[9]

**Imports.** Receipt line items and their semantics are imported from Synthesis 12.[3] ABOM/OINL packaging is imported from Anonymity C.[2] Transparency primitives (beacon/VRF binding and append-only logs with inclusion/consistency proofs) are imported from Synthesis 9.[5] Contract-object vocabulary is imported from Synthesis 0.[1] Trace-interface vocabulary (exposure normal forms) is imported from Synthesis 3.[4]

**Exports.** A minimal user-facing contract (Def. 1) and a validation lemma that connects it to the transparency-log interface.

## 1 The user-verifiability record

A UVI is a *small* object carried by the client (or shown by a UI) that points to a published receipt and the log proofs needed to trust that receipt.

**Definition 1** (User-verifiability record). *A user-verifiability record (UVI record) is a small object a client can retrieve while interacting with a deployment. It contains:*

1. **Receipt pointer (*envelope ids*):** *claim\_id*, *tw\_id*, and (when applicable) *state\_decl\_id*, matching the receipt envelope (Synthesis 12).[3]

2. **Surface identifier:** the declared `exposure_nf_id` naming the transcript projection the receipt covers.[4]
3. **Receipt digest:** a cryptographic digest of the exact receipt payload being relied on (ABOM/OINL carrier + line items).[2, 3]
4. **Log anchor:** a tuple sufficient to check inclusion in an append-only log: log id, signed checkpoint, and an inclusion proof for `receipt_digest`. [5]
5. **Consistency context:** either (i) a recent signed checkpoint previously trusted by the client, or (ii) a pointer to a consistency proof chain, so clients can detect equivocation across time.[5]

The record may be served via an HTTP endpoint, a signed gossip message, a DHT record, or a handshake extension; this note is agnostic to transport.

**Remark 1** (Why this is minimal). *The UVI record does not ship artifacts, evaluation traces, or proofs-of-correctness. It only lets a user answer three basic questions: “what receipt is the provider claiming?”, “what transcript surface does that receipt actually cover?”, and “is that receipt publicly anchored in an append-only log?” Those questions are sufficient to prevent a large class of silent drift, interface drift, and selective disclosure failures from being invisible to users.*

**Remark 2** (What a client can compare across releases). *A client does not need to understand the budget math to notice when a provider changes claim surface. If two releases advertise the same `claim_id` but a different `exposure_nf_id`, the client has learned something operationally important: the deployment is no longer presenting the same interface claim and should expose a new receipt lineage. Optional deployment-specific adjuncts may make that comparison easier (for example a watch policy or a compact compare profile), but those should remain adjacent public objects rather than extra mandatory UVI fields. The worked-example micro-note (Synthesis 17) shows one concrete way to pair a minimal UVI record with such adjuncts while keeping Def. 1 unchanged.[8]*

## 2 A validation lemma (what a user can check)

Let  $R$  be a receipt payload (ABOM/OINL carrier + line items), and let  $h(R)$  be the digest the UVI record presents. Let  $\mathcal{L}$  be an append-only log that supports (i) checkpointed signed tree heads and (ii) inclusion/consistency proofs.[5]

**Lemma 1** (User-side validation is mechanical). *Assume: (i) the receipt payload  $R$  is signed under the deployment’s declared release key (or notarization key), and (ii) the UVI record provides a valid inclusion proof that  $h(R)$  is included under a signed log checkpoint. Then a client can mechanically validate that (a) the provider is committing to  $R$  as a public claim object, and (b) the claim is anchored in an append-only log state that is auditable by third parties.*

*Proof sketch.* Verify the receipt signature using the declared release key. Verify the inclusion proof against the signed checkpoint. A standardized checklist for these mechanical steps (and for the auditor-side replay workflow) is given in Synthesis 20.[7] These checks are exactly the interface provided by the log primitive.[5] No privacy-domain reasoning is needed at validation time; semantics of the receipt line items are owned by their respective series notes and can be audited separately.[3]  $\square$

### 3 Failure modes (what this does *not* solve)

The UVI record is intentionally small; it does not eliminate all ways a provider can misbehave. We list common residual failure modes so mechanism papers do not have to repeat them.

**Withholding / non-discoverability.** A provider can simply refuse to serve a UVI record. This is a product and governance issue: UVI is meaningful only if clients treat absence as a policy failure.

**Equivocation across users.** A provider can serve different receipts to different users. Inclusion in an append-only log helps only if clients (or watchdogs) obtain and compare checkpoints and require consistency proofs. This is why Def. 1 includes *consistency context*.

**Compliance versus behavior.** A receipt is a claim; it does not force runtime compliance. Detecting runtime deviation requires monitoring, audits, and (often) closed-view attestations, which are treated elsewhere in the series.[6]

### 4 How this plugs into the series (one paragraph)

Synthesis 0 states a provider-side scope limitation: the archive is primarily about making claims addressable and auditable.[1] This note adds a minimal user-facing hook that preserves that philosophy: users need not re-derive anonymity proofs, but they can *pin* the provider to a public receipt, see which transcript surface that receipt covers, and detect silent drift or selective disclosure. Mechanism papers should not repeat UVI details; they should merely ensure their exported receipt line items carry an `exposure_nf_id`, can be carried by ABOM/OINL, and can be anchored by the transparency interface. When user-facing drift comparison matters, deployments may publish small adjunct objects around the UVI record, but the core discoverability contract should stay minimal.

## References

- [1] Anonymous. Anonymity Contract Objects for Anonymous DHTs: A Practical Interface for Proof-Carrying Claims. Series synthesis note (Synthesis 0), 2026. (This archive.)
- [2] Anonymous. ABOM and OINL for Anonymity Claims. Anonymity series Paper C, 2026. (This archive.)
- [3] Anonymous. Receipt Line Items for Anonymity Claims: A Minimal Tuple Schema for Published Budgets. Series synthesis note (Synthesis 12), 2026. (This archive.)
- [4] Anonymous. Trace Interfaces for Anonymous DHTs. Series synthesis note (Synthesis 3), 2026. (This archive.)
- [5] Anonymous. Transparency Primitives for Anonymity Receipts: Beacons, VRFs, and Append-Only Logs. Series synthesis note (Synthesis 9), 2026. (This archive.)
- [6] Anonymous. Closed-View Auditing and Proof-Carrying Privacy Cards (CPPC). AnonDHT State series Paper 3, 2026. (This archive.)
- [7] Anonymous. Auditor Replay for Anonymous-DHT Receipt Line Items: A Mechanical Checklist for Digest-Bound Claims. Series synthesis note (Synthesis 20), 2026. (This archive.)
- [8] Anonymous. Worked Example: Instantiating a Tiered Reader-Privacy Receipt. Synthesis 17, 2026. (This archive.)

- [9] T. Pauly and T. Reddy. *Discovery of Oblivious Services via Service Binding Records*. RFC 9540, IETF, February 2024. <https://www.rfc-editor.org/rfc/rfc9540>.